Day 2 – Lab 2.1: Timer-Triggered Stale Order Sweep

Azure Functions · C# · .NET 10 Isolated Worker · Flex Consumption

Objective: Build a scheduled Azure Function that executes a stale-order sweep every five minutes. Run it locally, deploy it to your independently created Day 2 Function App, and verify execution through Application Insights.

Business scenario: Contoso Orders needs a background process that periodically checks for orders requiring attention. In this lab, we simulate the scheduled sweep and validate its execution through logs. No database or external order-management system is required.

Prerequisites

Complete the Day 2 Participant Setup guide first.

Resource	Participant resource
Resource Group	Existing shared Resource Group
Function App	`fn-user01-orders-d2`
Storage Account	`stuser01ordersd2`

Runtime	.NET 10 isolated
Hosting plan	Flex Consumption
Local project	`contoso-fn`

Replace `user01` with your assigned participant ID.

1. Open the existing project

Open VS Code terminal:

```
None
cd contoso-fn
code .
```

Do not create a new Function App or Storage Account. Use the resources created in the Day 2 setup.

2. Install the Timer extension

Run:

```
None
dotnet add package
Microsoft.Azure.Functions.Worker.Extensions.Timer
dotnet restore
```

```
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> dotnet add package Microsoft.Azure.Functions.
sions.Timer
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.Azure.Functions.Worker.Extensions.Timer' into project 'C:
raft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.time
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.timer
419ms
info : Restoring packages for C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-order
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/index.jso
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/4.3.1/mic
.functions.worker.extensions.timer.4.3.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/4.3.1/micr
functions.worker.extensions.timer.4.3.1.nupkg 201ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/index.json 195ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/1.18.0/microsoft.azur
worker.core.1.18.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/1.18.0/microsoft.azure
orker.core.1.18.0.nupkg 203ms
info : Installed Microsoft.Azure.Functions.Worker.Extensions.Timer 4.3.1 from https://api.nuget.org/v3/index.js
rs\LabsKraft\.nuget\packages\microsoft.azure.functions.worker.extensions.timer\4.3.1 with content hash uNKWSuel
```

```
713yBkXZCkYnF/OcssXlONY+vM+AP+rm/wPR7iJ5xeDgOp5sOCKHEu5XYsbhZsfCRg==.
info : Installed Microsoft.Azure.Functions.Worker.Core 1.18.0 from https://api.nuget.org/v3/index.json to C:\Us
t\.nuget\packages\microsoft.azure.functions.worker.core\1.18.0 with content hash AFHJMjPB7JhwnWnGFbroSGMRa5ertG
3YmjrM29BasD9zL                                                                    /cKdVKQ==.
info :   CACHE  Follow link (ctrl + click)  v3/vulnerabilities/index.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.11.04.11.49.31/vulnerability.base.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.11.04.11.49.31/2026.11.05.11.49.38/vulnerability.u
info : Package 'Microsoft.Azure.Functions.Worker.Extensions.Timer' is compatible with all the specified framewo
ct 'C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info : PackageReference for package 'Microsoft.Azure.Functions.Worker.Extensions.Timer' version '4.3.1' added t
sers\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info : Writing assets file to disk. Path: C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\obj\pro
json
log  : Restored C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj (in 2
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> dotnet restore
Restore complete (1.6s)

Build succeeded in 3.3s
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders>
```

This installs Timer trigger support for the .NET isolated worker.

3. Create the Timer Function

Create a new file named:

`StaleOrderSweep.cs`

Add the following code:

None

```
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace ContosoFn;

public class StaleOrderSweep
{
    private readonly ILogger<StaleOrderSweep>
_logger;

    public StaleOrderSweep(ILogger<StaleOrderSweep>
logger)
    {
        _logger = logger;
    }

    [Function("StaleOrderSweep")]
    public void Run(
        [TimerTrigger("0 */5 * * * *")] TimerInfo
timer)
    {
        var utcTime = DateTime.UtcNow;

        _logger.LogInformation(
            "Stale order sweep started at: {Time}",
utcTime);

        if (timer.IsPastDue)
        {
```

```
            _logger.LogWarning(
                "Stale order sweep is running later
than scheduled.");
        }

        _logger.LogInformation(
            "Checking orders for stale status...");

        // Training simulation: no database is
connected.

        _logger.LogInformation(
            "Stale order sweep completed
successfully.");

        if (timer.ScheduleStatus?.Next != null)
        {
            _logger.LogInformation(
                "Next scheduled execution: {Next}",
                timer.ScheduleStatus.Next);
        }
    }
}
```

```
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> @'
>> using Microsoft.Azure.Functions.Worker;
>> using Microsoft.Extensions.Logging;
>>
>> namespace ContosoFn;
>>
>> public class StaleOrderSweep
>> {
>>     private readonly ILogger<StaleOrderSweep> _logger;
>>
>>     public StaleOrderSweep(ILogger<StaleOrderSweep> logger)
>>     {
>>         _logger = logger;
>>     }
>>
>>     [Function("StaleOrderSweep")]
>>     public void Run(
>>         [TimerTrigger("0 */5 * * * *")] TimerInfo timer)
>>     {
>>         var utcTime = DateTime.UtcNow;
>>
>>         _logger.LogInformation(
```

```
        _logger.LogInformation(
            "Stale order sweep started at: {Time}", utcTime);

        if (timer.IsPastDue)
        {
            _logger.LogWarning(
                "Stale order sweep is running later than scheduled.");
        }

        _logger.LogInformation(
            "Checking orders for stale status...");

        // Training simulation: no database is connected.

        _logger.LogInformation(
            "Stale order sweep completed successfully.");

        if (timer.ScheduleStatus?.Next != null)
        {
            _logger.LogInformation(
                "Next scheduled execution: {Next}",
```

```
>>                 timer.ScheduleStatus.Next);
>>         }
>>     }
>> }
>> '@ | Set-Content -Path "StaleOrderSweep.cs" -Encoding UTF8
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> Test-Path StaleOrderSweep.cs
True
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders>
```

What happens during execution?

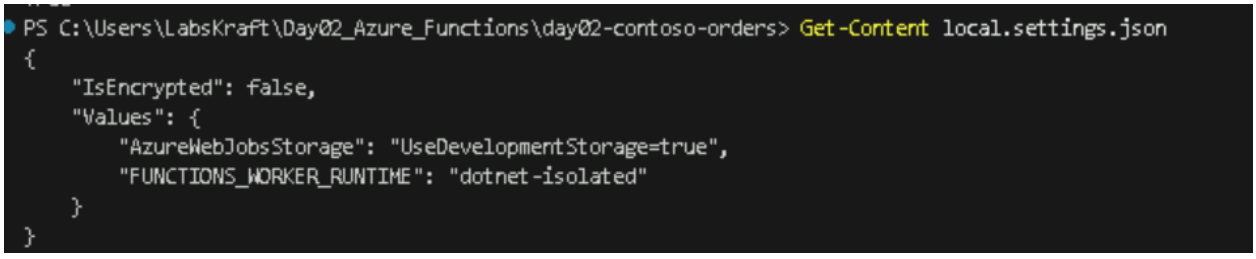
- Timer trigger starts the Function every five minutes.
- The Function records the UTC execution time.
- It checks whether the execution is late.
- It logs a simulated stale-order check.
- It records the next scheduled execution.

4. Configure local execution

Open `local.settings.json`.

Ensure the following settings are present:

```
None
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage":
"UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated"
  }
}
```



Start Azurite:

1. Open VS Code.
2. Press `Ctrl + Shift + P`.
3. Select `Azurite: Start`.

This allows the local Functions host to use the storage emulator. It does not change your Azure Function App's storage configuration.

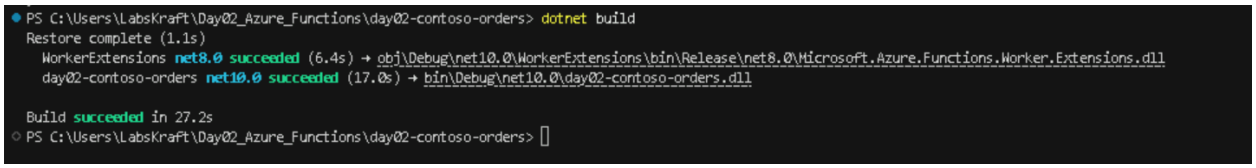
5. Build and run locally

Build the project:

```
None
dotnet build
```

Expected:

```
None
Build succeeded.
0 Error(s)
```

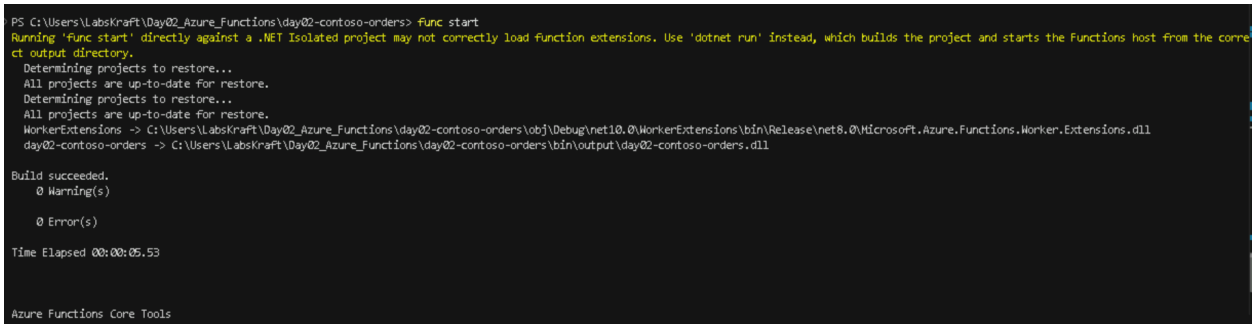


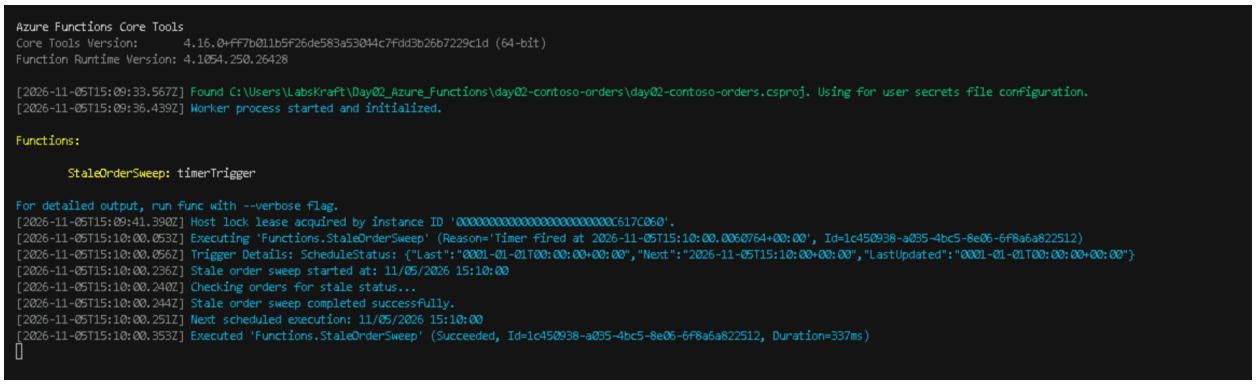
Start the Functions host:

```
None
func start
```

Expected Function listing:

```
None
StaleOrderSweep: timerTrigger
```





Wait for the next scheduled execution. The schedule runs every five minutes.

Expected logs:

```
Stale order sweep started at: <UTC timestamp>
Checking orders for stale status...
Stale order sweep completed successfully.
Next scheduled execution: <UTC timestamp>
```

Allow up to five minutes for the first scheduled execution. Do not add `RunOnStartup = true` for this exercise.

6. Understand the Timer schedule

Current expression:

```
None
[TimerTrigger("0 */5 * * * *")]
```

Azure Functions uses a six-field NCRONTAB expression.

Field	Value	Meaning
Seconds	`0`	At second zero
Minutes	`*/5`	Every five minutes
Hours	`*`	Every hour
Day	`*`	Every day
Month	`*`	Every month
Weekday	`*`	Every weekday

For a weekday schedule at 09:30 UTC, replace the expression with:

```
None
[TimerTrigger("0 30 9 * * 1-5")]
```

This runs Monday to Friday at 09:30 UTC.

For this lab, retain the original five-minute schedule.

7. Deploy to Azure

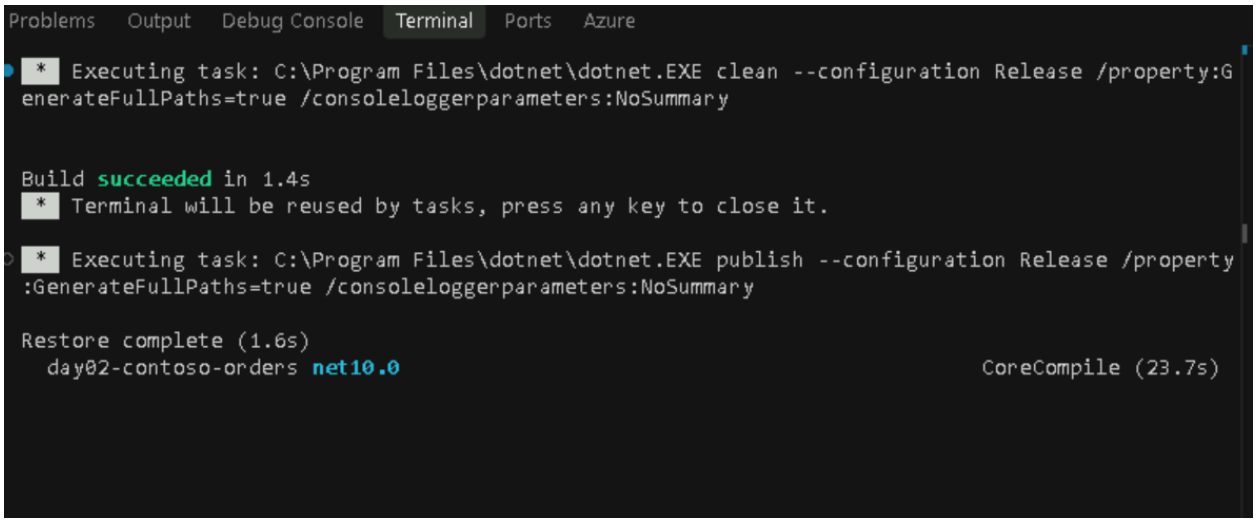
Stop the local Functions host using `Ctrl + C`.

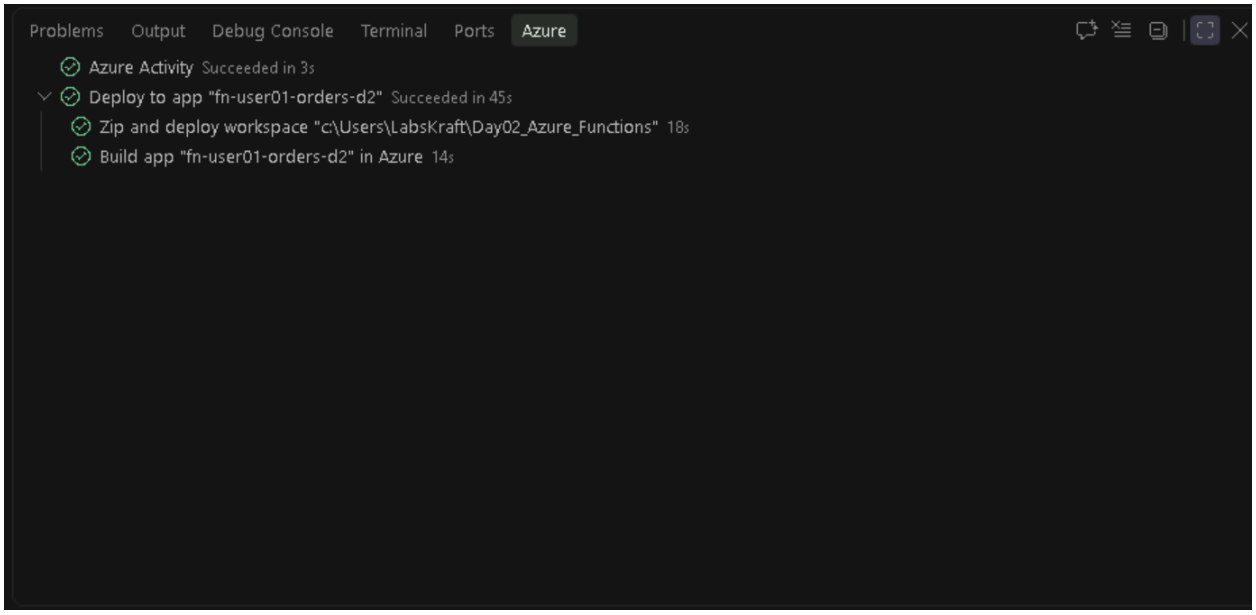
In VS Code:

1. Open the Azure extension.
2. Sign in to your Azure subscription.
3. Expand Resources → Function App.
4. Locate your Day 2 Function App:

```
None
fn-user01-orders-d2
```

5. Right-click the Function App.
6. Select Deploy to Function App.
7. Confirm the correct deployment target.
8. Wait for deployment to complete.

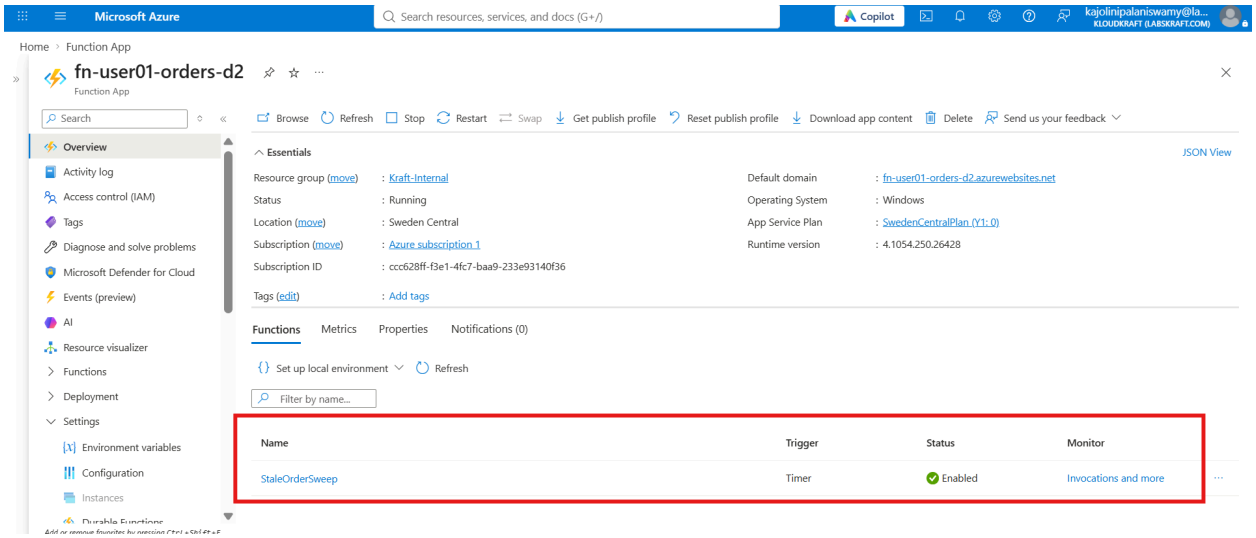




8. Validate execution in AzureNET

Open Azure Portal.

1. Open your Day 2 Function App.
2. Navigate to Functions.
3. Confirm `StaleOrderSweep` is listed.

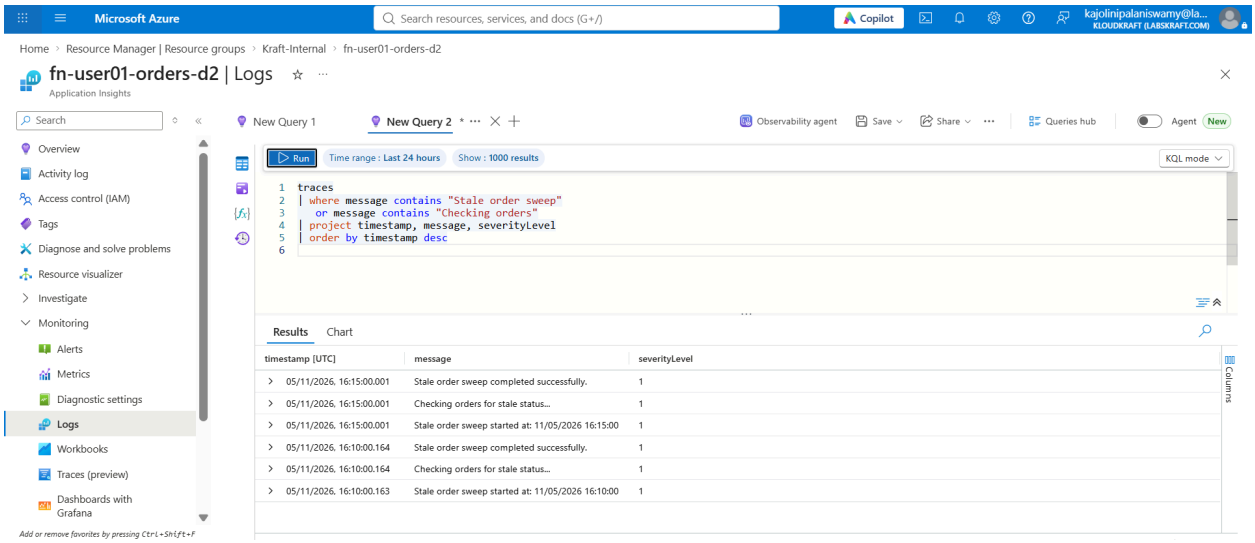


4. Open the Function App's Application Insights resource.
5. Select Logs.

Run:

```
None
traces
| where message contains "Stale order sweep"
   or message contains "Checking orders"
| project timestamp, message, severityLevel
| order by timestamp desc
```

Expected: Scheduled execution logs appear approximately every five minutes.



9. Troubleshooting

Issue	Check
Function not listed	Timer extension installed and project builds

Local Function does not execute	Azurite running and local storage setting correct
Azure Function does not execute	Function App running and Azure storage connection configured
Function shows errors	Review Application Insights exceptions and traces
Schedule appears incorrect	Verify six-field NCRONTAB expression and UTC time
Function is running late	Check host availability and storage connectivity

Production understanding: Timer Functions are useful for periodic background activities such as stale-order checks, reconciliation, scheduled reporting and cleanup. They avoid maintaining a continuously running server for scheduled tasks.

A Timer trigger uses storage-backed coordination within the Function App to prevent concurrent scheduled executions across scaled-out instances. It does not automatically retry a failed invocation like a Queue trigger.

10. Acceptance criteria

Validation	Expected result
Timer extension	Installed
Build	Successful
Local Function	`StaleOrderSweep` listed
Local execution	Logs appear every five minutes
Azure deployment	Successful
Azure Function	Visible in portal
Application Insights	Scheduled execution traces visible

11. Cleanup checkpoint

Keep resources for next labs

Do not delete Azure resources after Lab 2.1.

The same Day 2 Function App and Storage Account are required for Lab 2.3. Lab 2.2 will use Azurite locally.

Perform cleanup only after completing all three Day 2 labs, using the Day 2 participant cleanup guide.

Preserve:

- `contoso-fn` project.
- `StaleOrderSweep.cs`.
- Training datasets.
- Execution screenshots and evidence.

Lab 2.1 complete.

Next: Lab 2.2 – Blob → Queue → Blob using Azurite, retaining the existing project and keeping local testing independent of Azure resources.